

Quadro d'insieme

Che cos'è il lavoro fatto sul pagAmico, come ci si è arrivati, e dove siamo adesso

A cosa serve questo documento

È il primo dei cinque. Serve a rimettere in fila un lavoro che è cresciuto in sei giornate sparse e che adesso è fatto di trentanove file di codice: due librerie, sei programmi di prova, un analizzatore di log e i generatori di quattro documenti tecnici. Non spiega *come* funzionano le cose — per quello ci sono gli altri — ma **che cosa esiste, perché esiste e in che ordine è nato**.

I cinque documenti sono pensati per essere letti in quest'ordine, ma ognuno regge da solo:

	Documento	Risponde a
1	Quadro d'insieme (questo)	cos'è tutto questo, com'è nato, dove siamo
2	Le due librerie	che cosa possediamo, e che problemi risolve al posto nostro
3	I programmi di prova	a cosa serve ciascuno, e quando si apre quello e non un altro
4	Giano: dove siamo	cosa c'è già, cosa cambia col pagAmico, quanto lavoro è
5	Cosa non sappiamo	le domande aperte — è il documento per la telefonata

Sotto ci sono i due manuali tecnici già esistenti — *Integrazione pagAmico* e *Guida alle prove* — più l'analisi riga per riga dell'innesto in Giano. Quelli sono documenti di consultazione: si aprono quando serve un dettaglio, non si leggono di seguito.

1. Il problema di partenza

Il pagAmico di PayPrint è una cassa rendiresto che sta in rete. Ci si collega con un socket TCP, gli si scrivono dentro stringhe corte — `IN001550` significa "incassa 15,50 euro" — e lui risponde.

Il punto è che quel protocollo, visto da un'applicazione, non è una conversazione ordinata:

- non c'è un terminatore unico che dica dove finisce una risposta, e per i comandi il manuale non ne nomina nessuno: che sulla macchina vadano chiusi con CR o CR+LF l'ha scritto il fornitore solo l'11 settembre, e per i pacchetti immagine e la stampa diretta resta da chiedere;
- a un comando possono seguire tre o quattro risposte diverse, distanti minuti l'una dall'altra;
- alcune risposte sono JSON, altre sono testo nudo, e alcuni comandi non rispondono affatto;
- gli importi che mandi sono in centesimi, quelli che ricevi sono in euro;
- i nomi dei campi cambiano fra una versione di firmware e l'altra;
- e se mandi due comandi troppo ravvicinati il secondo viene ignorato **senza dire niente** — almeno sul simulatore, con i comandi senza terminatore. Per il fornitore, con CR o CR+LF non serve nessuna pausa. Dal 14 settembre le librerie chiudono ogni comando con CR, provato sul simulatore; la pausa di 80 ms resta finché la raffica con CR non regge su una macchina vera.

E soprattutto: **non esisteva niente da usare**. Nei quattro manuali di PayPrint l'unico esempio di codice è uno *snippet* Python di venti righe per l'invio di un'immagine; l'unico strumento che il fornitore suggerisce per parlare con il dispositivo è Hercules, un terminale TCP generico da scaricare da un sito terzo. Non c'è un SDK, non c'è un pacchetto. Il punto di partenza era una tabella di stringhe dentro un PDF.

Da qui la prima decisione. Scrivercele da soli, le librerie. Non è stata una scelta fra alternative: era l'unica strada. PayPrint stessa ci aveva detto che una libreria C# non esisteva.

2. Come ci si è arrivati

Sei giornate, in un arco di una settimana. La cronologia conta perché spiega perché certe cose sono fatte come sono fatte.

31 agosto — arrivano i manuali

Quattro PDF da PayPrint: il protocollo TCP-IP rev. 2.33, l'integrazione display 1.2, il protocollo di stampa 2.00 e le note di rilascio del firmware 8.72.

1 settembre — il primo nucleo, dai soli manuali

Trentanove minuti dopo il download parte il codice, e l'ordine dei file dice già molto: prima il **framing** — cioè come si ritagliano i messaggi dal flusso di byte — poi i comandi, poi il parser, poi i test. Non si è partiti dai comandi: si è partiti dal punto difficile.

Nove minuti dopo i test C# nascono i gemelli Kotlin. **La seconda libreria non è un porting fatto dopo: è una scelta presa in partenza.**

In questa fase il simulatore non esiste ancora. Tutto è ricavato dai PDF e verificato contro gli esempi letterali stampati nei manuali.

2 settembre, mattina — arriva il Dev Kit

pagAmico-DevKit-Setup-1.0.0.exe, con dentro un simulatore che apre un socket e risponde come il dispositivo vero, più una quinta guida che in diversi punti disambigua i quattro manuali.

È lo spartiacque del progetto. Da qui in avanti esiste una macchina con cui parlare.

2 settembre, pomeriggio — la giornata più densa

Alle 15:49 la prima sessione contro il simulatore. Alle 15:55 le prime due sessioni del proxy di analisi — ed è lì che salta fuori il primo difetto vero, l'avviso *"comando inviato solo 1 ms dopo il precedente"*. Il fornitore, nella risposta dell'11 settembre, dirà che nessuna pausa serve se i comandi terminano con CR o CR+LF, e che *"il simulatore probabilmente ha qualche difficoltà"*. Il 14 settembre, sul simulatore attuale, la raffica regge con e senza CR, e il CR diventa il default.

Nel giro di poche ore nascono il banco di prova, il collaudo automatico, il proxy, il programma di riempimento, e la prima generazione della documentazione. I due client grandi — il cuore delle librerie, quasi 70 KB in due — arrivano **per ultimi**, quando tutto il resto era già stato provato.

3 settembre — la giornata dell'usabilità

Nessun comando nuovo. Si costruisce il modo di *eseguire* le prove: la guida operativa, i profili di avvio numerati, le configurazioni per IntelliJ, il wrapper Gradle.

C'è una riga nei log di quel giorno che spiega tutto il capitolo: alle 16:43 compare logger agganciato a `--nologo:9100`. Un avvio da riga di comando in cui l'opzione è stata scambiata per il nome dell'host. Undici secondi dopo la riga è corretta.

Da lì la regola. Le prove non si fanno da terminale. Ogni programma che vuole argomenti ha un profilo salvato, numerato nell'ordine in cui si esegue: il menu a tendina dell'IDE è la sequenza delle prove.

7 settembre — la seconda ondata, e i documenti

Dopo tre giorni di pausa si riapre tutto. Questa volta la spinta è diversa: dal binario del Dev Kit è stato estratto il **catalogo comandi** dell'applicazione di PayPrint, che in diversi punti è più preciso dei manuali. Da lì si chiudono i tipi di barcode, si trova un esempio a dodici campi del comando `DI` — che resta però da confermare — e si aggiungono le sonde al collaudo.

Nel pomeriggio il lavoro passa dal banco di prova al gestionale: nasce l'analisi dell'innesto in Giano, e con essa **sei domande nuove** che finiscono nella mail a PayPrint. La risposta arriva l'11 settembre (`risposta-payprint-2026-09-11.md`): durante un incasso la macchina accetta `CM`, e il fornitore offre una telefonata e il collegamento alla sua macchina per le prove. La stessa risposta fa emergere tre difetti nelle librerie; l'analisi, verificata su codice, manuale e log, è in `esito-risposta-payprint.md`.

3. Le decisioni prese, e perché

Sette scelte che spiegano la forma del pacchetto.

3.1 Due librerie invece di una

Perché sono due piattaforme che nessun singolo binario copre: **C# per Giano**, che è WPF su .NET Framework, e **Kotlin per la parte Android**.

Ma c'è un beneficio secondario che vale quanto il primo, ed è quello da raccontare al telefono: **le due librerie si controllano a vicenda**. Generano le stesse identiche stringhe, e i test lo verificano contro gli esempi letterali dei manuali — **ottantasei asserzioni per parte**, identiche nei due linguaggi, a cui dall'11 settembre se ne aggiungono ottantadue sulle sequenze di incasso, sui comandi semplici, sul registro su file e sugli errori.

Due implementazioni indipendenti che convergono sulla stessa stringa sono una prova che la lettura del manuale è giusta. Una sola implementazione non prova niente: se hai capito male, hai capito male in un posto solo e nessuno se ne accorge.

3.2 Compilare per tre piattaforme invece che per una

La libreria C# dichiara `net8.0`, `netstandard2.0` e `net47`. Il primo serve ai banchi di prova e ai progetti nuovi, il secondo a qualunque altro consumatore, **il terzo a Giano**.

Questa decisione ha pagato in modo misurabile: quando è arrivato il momento di guardare l'innesto in Giano, è risultato che le dieci dipendenze della libreria sono **già tutte in Giano, alla versione identica**. Zero pacchetti nuovi, zero configurazioni da toccare. Basta prendere la DLL già compilata.

Il pezzo di codice che rende possibile tutto questo è di sedici righe: ridichiara un tipo che manca alle piattaforme vecchie, così lo stesso sorgente con la sintassi moderna compila anche per loro.

3.3 Un collaudo automatico oltre ai test

Perché i due provano cose diverse. I test offline confrontano le **stringhe generate** con gli esempi dei manuali: dimostrano che sappiamo comporre `IN001050`, non che la macchina lo accetti. E soprattutto non toccano la parte difficile, che è la **ricezione**.

La prova che serviva: **i primi due difetti veri del progetto li ha trovati il collaudo, non i test**. Nessuno dei due era visibile offline, perché entrambi riguardano il *tempo* — anche se il primo, i comandi ravvicinati, per il fornitore non si presenta se i comandi terminano con CR o CR+LF, cosa che le librerie oggi non fanno, e sul simulatore *probabilmente* c'entra una sua difficoltà: va provato. I tre venuti fuori dopo, con la risposta di PayPrint, il collaudo invece non li vedeva: passava lo stesso (D1-D3 in `esito-risposta-payprint.md`). D1 e D3 sono stati corretti l'11 settembre, con test offline di sequenza scritti apposta; D2 aspetta la macchina vera.

3.4 Un proxy che registra il traffico

Tre motivi, in ordine crescente di importanza. Il Dev Kit mostra il traffico a video ma non lo salva. Il proxy registra ogni segmento TCP **così com'è arrivato**, non il messaggio già ricomposto — ed è l'unico modo per vedere due comandi accorpati e per misurare le pause.

Il terzo motivo è quello che conta di più: **osservare l'applicazione di PayPrint mentre parla col simulatore**. Si punta il Dev Kit su una porta diversa, lui crede di parlare con la macchina, e intanto ogni byte finisce nel log. Quello che manda il loro programma era la risposta autorevole a metà delle domande

aperte. Terminatore e pause, per la macchina, li ha poi chiariti il fornitore per iscritto (resta da chiedere se il CR serve dopo i pacchetti immagine SF/SI, e come va con PTPRDT, che può contenere CR e LF); al proxy resta il formato dei comandi che il manuale non dà o dà in due modi, come DI e i pacchetti immagine.

3.5 Un programma per riempire il simulatore

Perché dopo qualche giro di collaudo il simulatore resta senza monete e non riesce più a dare resto, e metà dei passi fallirebbe per un motivo che non c'entra col codice.

La parte interessante è **perché non basta la ricarica prevista dal protocollo**: sul simulatore i comandi di ricarica aprono la sessione ma non entra nulla, perché sulla macchina vera il contante lo mette l'operatore. Quindi il programma riempie la macchina **eseguendo incassi**, alternando i tagli così che si ritrovi spiccioli di ogni valore e riesca a comporre qualsiasi resto.

3.6 Non implementare quello che non è confermato

Due comandi restano fuori perché il manuale li marca come deprecati. La stampa di immagini resta fuori perché il riepilogo la cita senza darne la sintassi. Il comando DI è implementato nella forma a sette campi, e la forma a dodici scoperta nel Dev Kit vive **solo come sonda nel collaudo**, non nell'API pubblica.

La disciplina è questa: l'esplorazione sta nel collaudo, l'interfaccia pubblica cambia solo su conferma. Vale la pena tenerla anche in seguito.

3.7 La documentazione è generata da codice

I due manuali tecnici non si scrivono a mano: si modifica uno script e si rilancia. È la ragione per cui la documentazione è rimasta allineata al codice attraverso sei giornate di lavoro invece di divergere dopo la prima.

4. Che cosa c'è adesso

I numeri

Albero	File	Righe
C# — la libreria	10	2.848
C# — banchi e strumenti (6 programmi)	10	3.078
Kotlin — la libreria (+ collaudo e test)	12	3.356
Kotlin — il banco di prova	3	1.247
Python — generatori di documentazione	3	2.351
Python — analizzatore dei log	1	466
Totale	39	13.346

Più circa 1.200 righe di documentazione in Markdown e due manuali PDF generati.

Il dato che dice più di tutti. Su quasi tredicimila righe, **le librerie vere sono 5.219** — il quaranta per cento. Tutto il resto — banchi, collaudo, proxy, riempimento, analisi dei log, generatori — è l'infrastruttura costruita per **dimostrare** che quelle 5.219 righe fanno la cosa giusta, contro una macchina che nessuno aveva in ufficio.

Lo stato della verifica

- **Test offline** tutti superati: **170 in C#, 171 in Kotlin**. 86 confrontano le stringhe generate con gli esempi letterali dei manuali, carattere per carattere; 63 in C# e 64 in Kotlin fanno parlare il client vero con un finto pagAmico su 127.0.0.1 (sequenze di incasso, comandi semplici, invio, keepalive); 21 provano il registro su file, anche con tre processi veri, e il vocabolario degli errori. Il test in più di Kotlin è proprio del linguaggio: l'esito di un annullato del chiamante viaggia con l'eccezione di cancellazione, mentre in C# viene restituito.
- **64 passi di collaudo** contro il simulatore, superati in entrambi i linguaggi. Sono i gruppi di default; accendendo anche la sonda del comando non documentato e i riavvii si arriva a 71. Dall'11 settembre il passo [IN]+[CM] dimostra davvero un commit: prima risultava OK leggendo l'accettazione (D1). Dei tre difetti venuti fuori con la risposta di PayPrint, D1 e D3 sono corretti; D2 resta aperto (esito-risposta-payprint.md).
- I due banchi di prova compilano e sono stati usati a mano su tutto il ciclo.
- **Nessuna prova su macchina fisica.** È il limite grosso. Il collegamento alla sua macchina l'ha offerto il fornitore stesso nella risposta dell'11 settembre, ma finché la prova non si fa il limite resta.

Che cosa è coperto e che cosa no

Il collaudo esercita **tutti** i comandi implementati: restano fuori solo i due deprecati. Ma è onesto sapere che i test offline coprono solo in parte **tempo, rete, stato e file**: dall'11 settembre le sequenze d'incasso e il registro su file sono provati contro un finto pagAmico, ma riconnessione, raffica e comportamento del dispositivo restano al collaudo, cioè a un simulatore acceso, e alla macchina vera.

5. Dove siamo, e che cosa manca

Il lavoro si è mosso su due piani, e adesso sono a punti diversi.

Il piano delle librerie sembrava sostanzialmente chiuso. Coprono il protocollo, sono collaudate, sono documentate, e hanno un'infrastruttura di prova che permette di rimetterle in discussione in dieci minuti quando serve. Ma la risposta di PayPrint ha fatto emergere tre difetti che il collaudo non vedeva. D1 e D3 — il commit letto sul messaggio sbagliato, l'attesa dell'incasso chiusa da un testo qualsiasi — sono corretti dall'11 settembre, senza macchina; D2, il timeout che abbandona un incasso ancora aperto, va con le prove su una macchina vera: il keepalive è regolato dal 14 settembre, manca la prova del cavo staccato (§3 dell'esito).

Il piano dell'innesto in Giano è appena cominciato, ed è fatto finora di sola analisi. La buona notizia è che la parte che sembrava più rischiosa — la compatibilità fra una libreria moderna e un progetto vecchio — si è rivelata un non-problema. La notizia meno buona è che il lavoro vero sta altrove: nel tradurre un modello di macchina in un altro, e nel decidere che cosa succede ai soldi quando qualcosa va storto.

Le tre cose che mancano, in ordine. Uno: le correzioni alle due librerie. PayPrint ha risposto l'11 settembre, e la domanda che bloccava il disegno del flusso di pagamento è chiusa: durante un incasso la macchina accetta CM. Il lavoro che resta adesso è nelle due librerie, elencato in ordine nel §3 di `esito-risposta-payprint.md`. Delle sei domande nate dall'innesto (nella mail 2.1-2.5 e 2.7) resta aperta la 2.2 e sono a metà la 2.3 e la 2.4: vanno alla telefonata con la 2.8, sulla password, e con quello che la risposta ha fatto emergere, a cominciare da quanti messaggi seguono un CM durante l'incasso (§5 dello stesso documento). La 1.1, sulla pausa, si chiude con le prove (§3, punti 9-11). **Due:** cinque prove, di cui tre vogliono una macchina vera — o almeno il collegamento alla sua, che il fornitore ha offerto. Dopo le sue risposte i parziali, CM durante l'incasso e l'incasso che il software abbandona diventano in parte una conferma, ma a ognuna resta qualcosa da scoprire: i campi dei parziali, quanti messaggi seguono un CM, che cosa arriva dopo una caduta di rete o una chiusura forzata. **Tre:** quattro decisioni contabili che spettano a noi e che nessuno può prendere leggendo il codice. Anche queste nel documento 5.

Le due domande tecniche da cui dipendeva la stima dell'adattatore — la 2.1 e la 2.7, CM durante l'incasso e i parziali — hanno avuto risposta, ma finché le decisioni contabili non sono prese la stima del lavoro su Giano non stringe sotto l'ordine di grandezza: **quindici-ventidue giornate** per la sostituzione alla pari, di cui circa un terzo è impalcatura a rischio basso e il resto è la parte dove si decide. Le correzioni alle librerie non ci sono dentro: vengono prima.

Il dettaglio di quella stima, con il precedente interno che le fa da taratura, sta nel documento 4.

6. Come si legge il resto

Se vuoi sapere	Apri
che cosa fa la libreria e che problemi risolve	documento 2, <i>Le due librerie</i>
il dettaglio tecnico del protocollo e del filo	<i>Integrazione pagAmico</i> (PDF)
a cosa serve ciascun programma di prova	documento 3, <i>I programmi di prova</i>
come si lancia una prova, passo per passo	<i>Guida alle prove</i> (PDF)
cosa c'è già in Giano e quanto lavoro è	documento 4, <i>Giano: dove siamo</i>
il dettaglio riga per riga dell'innesto	<code>analisi-giano-vne-vs-pagamico.md</code>
che cosa chiedere a PayPrint e cosa decidere	documento 5, <i>Cosa non sappiamo</i>
il testo della mail e la risposta di PayPrint	<code>mail-payprint-domande-protocollo.md</code> , <code>risposta-payprint-2026-09-11.md</code>
che cosa cambia dopo la risposta, verificato sul codice	<code>esito-risposta-payprint.md</code>